

Complexity Analysis of Representative Cryptographic Protocols:

Zero-Knowledge Proofs, Oblivious Transfer, and Lattice-Based Encryption

Sayed Mohammad Badiehzadegan

Independent Cryptography and Quantum Computing Researcher
admin@quantize.ca
Richmond, British Columbia, Canada

August 2, 2026

Abstract

Cryptographic protocols are evaluated not only by the security properties they achieve, but also by the resources required to execute them. Unlike conventional algorithm analysis, where a single running-time function may be sufficient, protocol analysis usually separates computational complexity, bit complexity, communication complexity, storage complexity, and round complexity. This article develops a systematic methodology for such analysis through three representative case studies: the Schnorr zero-knowledge proof of knowledge, a Diffie–Hellman-based one-out-of-two oblivious-transfer protocol, and Regev-style public-key encryption based on the Learning With Errors problem. For each construction, the protocol is stated formally, correctness is derived, and the dominant computational, communication, and memory costs are calculated step by step. The examples illustrate three qualitatively different bottlenecks: group exponentiation in discrete-logarithm protocols, amortized public-key cost in oblivious transfer, and dense matrix–vector arithmetic in lattice-based cryptography.

Keywords: zero-knowledge proof, Schnorr protocol, oblivious transfer, OT extension, lattice-based cryptography, Learning With Errors, complexity analysis, communication complexity.

1 Introduction

A cryptographic construction must satisfy two broad requirements. First, it must provide a rigorously defined security guarantee, such as secrecy, authentication, zero knowledge, indistinguishability, or resistance to a specified adversarial model. Second, it must be computationally and communicationally feasible for the environment in which it is deployed.

These requirements are related but distinct. A protocol may be provably secure while requiring impractically large keys, excessive communication, or too many rounds. Conversely, an efficient protocol is unsuitable if its security argument is incomplete or based on unrealistic assumptions. Consequently, the study of modern cryptographic protocols combines mathematical security analysis with systematic resource analysis.

For an ordinary sequential algorithm, one often summarizes efficiency by a running-time function such as

$$O(n), \quad O(n \log n), \quad O(n^2).$$

For an interactive cryptographic protocol, however, a more informative cost model is

protocol cost = computation + communication + storage + rounds.

Each component may depend on a security parameter, a message length, a group size, a matrix dimension, or the number of protocol participants.

This article develops this methodology through three examples:

- (i) the Schnorr proof of knowledge of a discrete logarithm;
- (ii) a Diffie–Hellman-based one-out-of-two oblivious transfer;
- (iii) Regev-style encryption based on Learning With Errors (LWE).

These examples were selected because they exhibit different algebraic structures and different resource bottlenecks. Schnorr is dominated by group exponentiation or elliptic-curve scalar multiplication. Oblivious transfer combines public-key operations with message-dependent symmetric operations. Regev encryption is dominated by modular linear algebra and illustrates the large-key phenomenon of basic lattice constructions.

2 A Framework for Cryptographic Complexity Analysis

2.1 Security parameters and input parameters

Let λ denote a generic security parameter. Depending on the construction, the concrete parameters may include:

- $k = \lceil \log_2 q \rceil$, the bit length of a group order or modulus;
- ℓ , the number of bits in an application message;
- n , a lattice or secret-vector dimension;
- m , the number of LWE samples;
- N , the number of independent protocol instances.

A complete analysis should state which quantities are regarded as variable. For example, saying that a protocol performs a constant number of exponentiations does not mean that it runs in constant bit time: the cost of each exponentiation grows with the security parameter.

2.2 Algebraic-operation complexity

Cryptographic papers frequently count expensive algebraic operations rather than elementary machine instructions. Typical units include:

$$\text{Exp}, \quad \text{Mul}, \quad \text{Add}, \quad \text{Hash}, \quad \text{XOR}.$$

This style is useful because the dominant operations depend strongly on the chosen platform and implementation. For example, the cost of one elliptic-curve scalar multiplication may dominate dozens of hash evaluations, while modular additions are comparatively inexpensive.

If a prover performs one exponentiation, one modular multiplication, and one modular addition, it is informative to write

$$T_P = 1\text{Exp} + 1\text{Mul} + 1\text{Add}.$$

Such an expression preserves practical information that may be hidden by a coarse asymptotic bound.

2.3 Bit complexity

To reduce algebraic operations to bit complexity, let $M(k)$ denote the time required to multiply two k -bit integers. With schoolbook multiplication,

$$M(k) = O(k^2).$$

A modular exponentiation $g^x \bmod p$, with a k -bit exponent, requires $O(k)$ modular multiplications under square-and-multiply. Therefore,

$$T_{\text{exp}}(k) = O(kM(k)).$$

Under schoolbook multiplication,

$$T_{\text{exp}}(k) = O(k^3).$$

This estimate is intentionally simplified. Faster multiplication algorithms, windowed exponentiation, Montgomery reduction, fixed-base precomputation, and elliptic-curve optimizations can improve concrete performance. Nevertheless, the model is adequate for deriving transparent upper bounds.

2.4 Communication, space, and round complexity

Communication complexity counts the total number of transmitted bits:

$$C(\lambda) = \sum_i |M_i|,$$

where M_i is the i th protocol message.

Space complexity measures either total storage or peak working memory. Round complexity counts sequential dependencies between messages. A protocol with linear communication but many sequential rounds may perform poorly over a high-latency network, whereas a protocol with larger messages but fewer rounds may be preferable.

3 Case Study I: The Schnorr Zero-Knowledge Proof

3.1 Objective and setting

Let $G = \langle g \rangle$ be a cyclic group of prime order q . The prover possesses a secret

$$x \in \mathbb{Z}_q$$

and publishes

$$y = g^x.$$

The prover wishes to convince a verifier that it knows x without revealing the witness itself. The Schnorr identification protocol is a canonical three-move proof of knowledge based on the discrete-logarithm relation [?].

3.2 Protocol description

The protocol consists of commitment, challenge, and response.

Commitment. The prover samples

$$r \xleftarrow{\$} \mathbb{Z}_q$$

and computes

$$t = g^r.$$

It sends t to the verifier.

Challenge. The verifier samples

$$c \xleftarrow{\$} \mathbb{Z}_q$$

and sends c to the prover.

Response. The prover computes

$$s = r + cx \pmod{q}$$

and sends s .

Verification. The verifier accepts if

$$g^s = t y^c.$$

3.3 Correctness

For an honest prover,

$$\begin{aligned} g^s &= g^{r+cx} \\ &= g^r g^{cx} \\ &= g^r (g^x)^c \\ &= t y^c. \end{aligned}$$

Therefore every honestly generated transcript is accepted.

3.4 Prover computation

The prover performs:

1. one group exponentiation $t = g^r$;
2. one modular multiplication $cx \bmod q$;
3. one modular addition $r + cx \bmod q$.

Hence,

$$T_P = 1\text{Exp} + 1\text{Mul} + 1\text{Add}.$$

Exponentiation dominates the scalar arithmetic, so

$$\boxed{T_P = O(\text{Exp}(k)).}$$

Under the simplified schoolbook model,

$$\boxed{T_P = O(k^3).}$$

If an elliptic-curve group is used, exponentiation is replaced by scalar multiplication. The algebraic count remains one dominant group operation, although its implementation differs.

3.5 Verifier computation

The verifier checks

$$g^s = t y^c.$$

A direct evaluation requires:

- one exponentiation g^s ;
- one exponentiation y^c ;
- one group multiplication.

Therefore,

$$T_V = 2\text{Exp} + 1\text{GroupMul},$$

and

$$T_V = O(\text{Exp}(k)) = O(k^3)$$

under the same simplified bit model.

In practice, simultaneous multi-exponentiation can evaluate the verification equation more efficiently than two entirely independent exponentiations, but the asymptotic class is unchanged.

3.6 Communication complexity

The transmitted objects are

$$t, \quad c, \quad s.$$

Let L_G be the encoded length of one group element. The challenge and response are elements of $\mathbb{Z}_q$, requiring approximately k bits each. Thus,

$$C_{\text{Schnorr}} = L_G + 2k.$$

For common prime-order groups, $L_G = \Theta(k)$, yielding

$$C_{\text{Schnorr}} = O(k).$$

The protocol consists of three messages. Equivalently, it is a three-move public-coin protocol with a constant number of communication rounds.

3.7 Space complexity

The prover stores a constant number of scalars and group elements:

$$x, \quad r, \quad t, \quad c, \quad s.$$

Consequently,

$$S_P = O(k).$$

The verifier also stores only a constant number of $O(k)$ -bit objects:

$$S_V = O(k).$$

3.8 Soundness amplification

When the challenge is selected from all of $\mathbb{Z}_q$, the knowledge error is approximately $1/q$. In simplified pedagogical variants with a one-bit challenge, a cheating prover can answer at most one of two possible challenges for a fixed commitment, giving success probability at most

$$\frac{1}{2}$$

per execution.

After r independent repetitions,

$$P_{\text{cheat}} \leq 2^{-r}.$$

To achieve λ bits of soundness, one may choose

$$r = \lambda.$$

Sequential repetition gives

$$T = O(\lambda \text{Exp}(k)), \quad C = O(\lambda k), \quad R = O(\lambda).$$

Parallel repetition can reduce the sequential round count, though the total arithmetic and communication remain proportional to the number of repetitions.

4 Case Study II: One-Out-of-Two Oblivious Transfer

4.1 Functionality

In a one-out-of-two oblivious transfer, the sender holds two messages

$$m_0, m_1 \in \{0, 1\}^\ell,$$

and the receiver holds a choice bit

$$b \in \{0, 1\}.$$

At the end of the protocol:

1. the receiver learns m_b ;
2. the receiver should not learn meaningful information about m_{1-b} ;
3. the sender should not learn b .

Oblivious transfer is a fundamental primitive for secure multiparty computation. The following construction captures the algebraic core of the efficient Diffie–Hellman-based approach associated with Chou and Orlandi [?]. For formal deployment, the exact protocol specification, group checks, hash-domain separation, and security model must be followed.

4.2 Simplified protocol

Let $G = \langle g \rangle$ be a cyclic group of prime order q , and let

$$H : G \rightarrow \{0, 1\}^\ell$$

be a hash or key-derivation function.

Sender initialization. The sender samples

$$a \xleftarrow{\$} \mathbb{Z}_q$$

and sends

$$A = g^a.$$

Receiver message. The receiver samples

$$k \xleftarrow{\$} \mathbb{Z}_q.$$

It constructs

$$B = \begin{cases} g^k, & b = 0, \\ Ag^k, & b = 1, \end{cases}$$

and sends B .

Sender key derivation. The sender derives

$$K_0 = H(B^a)$$

and

$$K_1 = H((B/A)^a).$$

It forms

$$e_0 = m_0 \oplus K_0, \quad e_1 = m_1 \oplus K_1,$$

and sends (e_0, e_1) .

Receiver recovery. The receiver computes

$$K_b = H(A^k)$$

and obtains

$$m_b = e_b \oplus K_b.$$

4.3 Correctness

If $b = 0$, then $B = g^k$, and

$$B^a = (g^k)^a = g^{ak} = A^k.$$

Therefore, the receiver and sender derive the same K_0 .

If $b = 1$, then $B = Ag^k$, so

$$\frac{B}{A} = g^k$$

and

$$(B/A)^a = g^{ak} = A^k.$$

Hence the receiver and sender derive the same K_1 .

4.4 Sender computation

The sender performs approximately:

1. one exponentiation $A = g^a$;
2. one exponentiation B^a ;
3. one exponentiation $(B/A)^a$;
4. a constant number of group multiplications or inversions;
5. two hash/KDF evaluations;
6. two XOR operations on ℓ -bit strings.

Thus,

$$T_S = 3\text{Exp} + 2\text{Hash} + 2\text{XOR}_\ell + O(1) \text{ group operations.}$$

Asymptotically,

$$T_S = O(\text{Exp}(k) + T_H(\ell) + \ell).$$

Under schoolbook modular arithmetic,

$$T_S = O(k^3 + T_H(\ell) + \ell).$$

4.5 Receiver computation

The receiver performs approximately:

1. one exponentiation g^k ;
2. possibly one group multiplication to form Ag^k ;
3. one exponentiation A^k ;
4. one hash/KDF evaluation;
5. one XOR on an ℓ -bit string.

Therefore,

$$T_R = 2\text{Exp} + 1\text{Hash} + 1\text{XOR}_\ell + O(1) \text{ group operations,}$$

and

$$T_R = O(\text{Exp}(k) + T_H(\ell) + \ell).$$

4.6 Communication complexity

The protocol transmits:

- A , one group element;
- B , one group element;
- e_0 and e_1 , two ℓ -bit ciphertext strings.

Therefore,

$$C_{\text{OT}} = 2L_G + 2\ell.$$

If $L_G = \Theta(k)$,

$$C_{\text{OT}} = O(k + \ell).$$

The sequence

$$A \longrightarrow B \longrightarrow (e_0, e_1)$$

contains three messages and a constant number of communication rounds:

$$R_{\text{OT}} = O(1).$$

4.7 Space complexity

The sender stores two input messages and a constant number of cryptographic values:

$$S_S = O(k + \ell).$$

The receiver stores its choice, one recovered message, and temporary group elements:

$$S_R = O(k + \ell).$$

4.8 Repeated oblivious transfers

Suppose an application requires N independent oblivious transfers. Executing the public-key protocol independently gives

$$T = O(N \text{Exp}(k) + N\ell)$$

and

$$C = O(N(k + \ell)).$$

The dominant cost is the repetition of public-key operations.

OT extension reduces this cost by using a relatively small number of expensive base OTs and then generating many additional OTs using mostly symmetric-key operations. The seminal IKNP construction formalized this approach in the random-oracle model [?]. At a high level, one replaces

N public-key OTs

with

κ base OTs + $O(N\kappa)$ inexpensive bit and hash operations,

where κ is a computational security parameter. When $N \gg \kappa$, the amortized public-key cost per OT becomes small.

This example demonstrates why asymptotic analysis should sometimes be performed over batches rather than individual protocol executions. A base OT may be expensive in isolation but efficient as a setup phase for a large secure computation.

5 Case Study III: Regev-Style LWE Encryption

5.1 The Learning With Errors setting

Let:

- n be the secret dimension;
- m be the number of samples;

- q be an integer modulus;
- χ be a distribution concentrated on small errors.

Choose

$$\mathbf{s} \in \mathbb{Z}_q^n, \quad A \in \mathbb{Z}_q^{m \times n}, \quad \mathbf{e} \leftarrow \chi^m.$$

Define

$$\mathbf{b} = A\mathbf{s} + \mathbf{e} \pmod{q}.$$

The public key is

$$pk = (A, \mathbf{b}),$$

and the secret key is

$$sk = \mathbf{s}.$$

The LWE problem asks, in one formulation, whether noisy linear samples can be distinguished from uniform samples or whether the hidden secret can be recovered. Regev established a connection between LWE and worst-case lattice problems and presented a public-key cryptosystem based on this hardness assumption [?, ?].

5.2 Encryption

To encrypt a bit

$$\mu \in \{0, 1\},$$

sample

$$\mathbf{r} \xleftarrow{\$} \{0, 1\}^m.$$

Compute

$$\mathbf{u} = A^T \mathbf{r} \pmod{q}$$

and

$$v = \mathbf{b}^T \mathbf{r} + \mu \left\lfloor \frac{q}{2} \right\rfloor \pmod{q}.$$

The ciphertext is

$$ct = (\mathbf{u}, v).$$

5.3 Decryption

The receiver computes

$$d = v - \mathbf{s}^T \mathbf{u} \pmod{q}.$$

Substituting the definitions,

$$\begin{aligned} d &= \mathbf{b}^T \mathbf{r} + \mu \left\lfloor \frac{q}{2} \right\rfloor - \mathbf{s}^T A^T \mathbf{r} \\ &= (A\mathbf{s} + \mathbf{e})^T \mathbf{r} + \mu \left\lfloor \frac{q}{2} \right\rfloor - \mathbf{s}^T A^T \mathbf{r} \\ &= \mathbf{e}^T \mathbf{r} + \mu \left\lfloor \frac{q}{2} \right\rfloor \pmod{q}. \end{aligned}$$

If the accumulated error $\mathbf{e}^T \mathbf{r}$ is sufficiently small relative to q , values near 0 decode to $\mu = 0$, while values near $q/2$ decode to $\mu = 1$.

5.4 Key-generation complexity

The dominant operation is

$$As.$$

The matrix A contains mn coefficients. Each of the m output entries is a dot product of length n . Therefore, key generation performs:

$$mn$$

modular multiplications and approximately

$$m(n-1)$$

modular additions.

Thus,

$$T_{\text{KeyGen}} = O(mn)$$

modular operations.

Let

$$L_q = \lceil \log_2 q \rceil.$$

If modular multiplication costs $M(L_q)$, then

$$T_{\text{KeyGen}} = O(mn M(L_q)).$$

With schoolbook multiplication,

$$T_{\text{KeyGen}} = O(mn \log^2 q).$$

Sampling the public matrix also requires generating mn residues modulo q , and sampling the error vector requires m draws from χ .

5.5 Encryption complexity

The primary encryption operation is

$$\mathbf{u} = A^T \mathbf{r}.$$

Since A^T has dimensions $n \times m$, a dense implementation processes nm matrix coefficients. The scalar

$$v = \mathbf{b}^T \mathbf{r} + \mu \lfloor q/2 \rfloor$$

requires an additional m coefficient operations.

Therefore,

$$T_{\text{Enc}} = O(nm + m) = O(nm)$$

modular operations, and

$$T_{\text{Enc}} = O(nm M(\log q)).$$

Using schoolbook multiplication,

$$T_{\text{Enc}} = O(nm \log^2 q).$$

Because $\mathbf{r}$ is binary, multiplication by $r_i \in \{0, 1\}$ can be implemented as a conditional inclusion of a matrix row rather than a general modular multiplication. This may substantially improve the concrete constant factor, but a dense implementation still examines $O(nm)$ entries.

5.6 Decryption complexity

Decryption evaluates the inner product

$$\mathbf{s}^T \mathbf{u}.$$

The vectors have length n , so the computation uses:

- n modular multiplications;
- approximately $n - 1$ modular additions;
- one subtraction;
- one threshold decision.

Thus,

$$T_{\text{Dec}} = O(n)$$

modular operations, or

$$T_{\text{Dec}} = O(n M(\log q)).$$

With schoolbook multiplication,

$$T_{\text{Dec}} = O(n \log^2 q).$$

The basic scheme therefore has the qualitative relationship

$$T_{\text{KeyGen}} \approx T_{\text{Enc}} \gg T_{\text{Dec}}.$$

5.7 Key and ciphertext sizes

The public matrix contains mn residues modulo q , and $\mathbf{b}$ contains m residues. Hence,

$$|pk| = (mn + m)L_q,$$

so

$$|pk| = O(mn \log q).$$

The secret key contains n residues:

$$|sk| = nL_q,$$

and therefore

$$|sk| = O(n \log q).$$

The ciphertext contains $\mathbf{u} \in \mathbb{Z}_q^n$ and one element $v \in \mathbb{Z}_q$, yielding

$$|ct| = (n + 1)L_q,$$

or

$$|ct| = O(n \log q)$$

for the encryption of one plaintext bit in this simplified construction.

Regev's original parameterization yields a public key of approximately $\tilde{O}(n^2)$ bits and ciphertext expansion of approximately $\tilde{O}(n)$ for a single-bit message, where $\tilde{O}$ suppresses logarithmic factors [?, ?].

5.8 Space complexity

An explicit implementation stores the full public matrix:

$$S_{\text{KeyGen}} = O(mn \log q).$$

Encryption likewise requires access to A :

$$S_{\text{Enc}} = O(mn \log q)$$

if the matrix is materialized.

Memory can be reduced by generating matrix entries pseudorandomly from a compact seed and processing them incrementally. Such a technique reduces peak storage but does not remove the need to process approximately mn coefficients.

Decryption stores vectors of length n :

$$S_{\text{Dec}} = O(n \log q).$$

6 Numerical Illustration for the LWE Construction

Consider the educational parameter set

$$n = 256, \quad m = 4096, \quad q = 12289.$$

These values are used only to illustrate resource calculations and do not constitute a recommended secure parameter set.

Since

$$\log_2(12289) \approx 13.6,$$

each coefficient can be represented using approximately 14 bits.

6.1 Matrix size

The matrix contains

$$mn = 256 \times 4096 = 1,048,576$$

coefficients. Its ideal packed size is approximately

$$1,048,576 \times 14 = 14,680,064 \text{ bits.}$$

Equivalently,

$$\frac{14,680,064}{8} = 1,835,008 \text{ bytes} \approx 1.75 \text{ MiB.}$$

This estimate excludes metadata, alignment, and implementation overhead. If each coefficient is stored in a 16-bit integer, the matrix requires approximately 2 MiB.

6.2 Encryption workload

The product

$$A^T \mathbf{r}$$

processes approximately

$$1,048,576$$

matrix coefficients. Since $\mathbf{r}$ is binary, this may correspond mainly to conditional modular additions rather than full multiplications. Nevertheless, encryption is on the order of one million coefficient-processing operations for this dense example.

6.3 Decryption workload

Decryption computes a dot product of length 256. It therefore requires approximately

256

modular multiplications and

255

modular additions, followed by one modular subtraction and a decoding comparison. This large difference illustrates why decryption is much cheaper than encryption in the basic construction.

7 Comparative Analysis

Table ?? summarizes the dominant costs.

Table 1: Asymptotic comparison of the three representative constructions.

Protocol	Dominant initiator cost	Dominant responder cost	re-	Communication	Rounds
Schnorr ZK	One exponentiation plus scalar arithmetic	Two exponentiations plus one group multiplication		$O(k)$	$O(1)$
DH-based OT	Constant exponentiations plus $O(\ell)$ hashing/XOR	Constant exponentiations plus $O(\ell)$ hashing/XOR		$O(k + \ell)$	$O(1)$
Regev LWE encryption	$O(mn)$ modular co-efficient operations	$O(n)$ modular operations for decryption		Ciphertext $O(n \log q)$	Noninteractive

The protocols exhibit three distinct bottlenecks.

Schnorr. Its cost is dominated by a constant number of group exponentiations or scalar multiplications. Its communication and storage are linear in the security parameter, and its number of rounds is constant.

Oblivious transfer. A single instance also requires only a constant number of public-key operations, but large secure-computation applications may require millions of OTs. Therefore, amortized batch complexity is more important than the cost of one instance. OT extension transforms the dominant workload from repeated public-key cryptography into symmetric-key operations.

LWE encryption. The basic construction is dominated by dense matrix–vector arithmetic. Its public key can be much larger than those of traditional discrete-logarithm systems, and encryption processes $O(mn)$ coefficients. Decryption is considerably cheaper, requiring only a length- n inner product.

8 Methodological Lessons

The three case studies support several general conclusions.

8.1 Operation counts and asymptotic notation are complementary

Writing

$$O(k^3)$$

for both a prover and verifier can conceal the difference between one exponentiation and two exponentiations. For practical comparison, algebraic-operation counts should be reported together with asymptotic bit bounds.

8.2 Communication depends on representation

The statement $L_G = \Theta(k)$ is asymptotically correct for conventional prime-order groups, but concrete sizes depend on the chosen representation, point compression, field size, and encoding rules. Similarly, lattice coefficient storage depends on whether coefficients are tightly packed or stored in fixed-width machine words.

8.3 Round complexity is independent of total bandwidth

A protocol can have low total communication and still perform poorly on a high-latency network if it requires many sequential messages. Conversely, a constant-round protocol may transmit large objects but remain suitable for distributed execution.

8.4 Batch complexity may determine real performance

The OT example shows that analyzing one protocol invocation may be misleading. When a primitive is invoked repeatedly, preprocessing and amortization can change the dominant complexity. The relevant quantity may be

$$\frac{\text{total cost for } N \text{ executions}}{N}$$

rather than the cost of a standalone execution.

8.5 Memory and computation may be traded

The LWE matrix can be stored explicitly or generated from a seed. Explicit storage reduces repeated generation but consumes $O(mn \log q)$ bits. Seeded generation reduces memory but introduces pseudorandom-generation work and may alter data-access patterns. Such time-space tradeoffs are central to bounded-space and implementation-oriented cryptography.

9 Conclusion

Complexity analysis of cryptographic protocols requires a multidimensional resource model. Computational complexity alone does not capture communication volume, storage requirements, or the latency introduced by sequential rounds. Moreover, an asymptotic expression should normally be supplemented by counts of the dominant cryptographic operations.

The Schnorr protocol demonstrates a compact zero-knowledge proof whose dominant costs are a constant number of group exponentiations. Diffie–Hellman-based oblivious transfer illustrates the combination of public-key and message-length-dependent symmetric operations, as well as the importance of amortization through OT extension. Regev-style LWE encryption demonstrates a different computational profile, dominated by matrix–vector arithmetic and comparatively large public data.

Together, these examples provide a reusable procedure:

1. identify all variable security and input parameters;

2. write each protocol step as an algebraic operation;
3. count dominant operations separately for each party;
4. translate those counts into bit complexity when useful;
5. calculate message sizes and sequential rounds;
6. calculate peak storage and identify time–space tradeoffs;
7. analyze batching, repetition, and amortization where applicable.

This methodology is applicable well beyond the three protocols considered here. It can be extended to multiparty computation, post-quantum signatures, homomorphic encryption, private set intersection, verifiable computation, and other foundational cryptographic constructions.

References

- [1] C. P. Schnorr, “Efficient Signature Generation by Smart Cards,” *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991. doi: [10.1007/BF00196725](https://doi.org/10.1007/BF00196725).
- [2] T. Chou and C. Orlandi, “The Simplest Protocol for Oblivious Transfer,” in *Progress in Cryptology–LATINCRYPT 2015*, Lecture Notes in Computer Science, vol. 9230, Springer, pp. 40–58, 2015. doi: [10.1007/978-3-319-22174-8_3](https://doi.org/10.1007/978-3-319-22174-8_3). Preprint: <https://eprint.iacr.org/2015/267>.
- [3] Y. Ishai, J. Kilian, K. Nissim, and E. Petrank, “Extending Oblivious Transfers Efficiently,” in *Advances in Cryptology–CRYPTO 2003*, Lecture Notes in Computer Science, vol. 2729, Springer, pp. 145–161, 2003. doi: [10.1007/978-3-540-45146-4_9](https://doi.org/10.1007/978-3-540-45146-4_9).
- [4] O. Regev, “On Lattices, Learning with Errors, Random Linear Codes, and Cryptography,” in *Proceedings of the 37th Annual ACM Symposium on Theory of Computing*, pp. 84–93, 2005. doi: [10.1145/1060590.1060603](https://doi.org/10.1145/1060590.1060603).
- [5] O. Regev, “On Lattices, Learning with Errors, Random Linear Codes, and Cryptography,” *Journal of the ACM*, vol. 56, no. 6, Article 34, 2009. doi: [10.1145/1568318.1568324](https://doi.org/10.1145/1568318.1568324).